

NAME : DANIYA

ROLL NO : CT-25159

COURSE CODE: CT-159

INSTRUCTOR : MUDASSIR AHMED

LAB 01

QUESTION NO 1:

SOURCE CODE:

```
ques1.cpp converted(const vector<vector<int>>& arr2D, vector<int>& arr1D, int rows, int cols) {
1  #include<bits/stdc++.h>
2  using namespace std;
3
4  // Function to copy a 2D array into a 1D array using Column Major Order
5  void convert2D(const vector<vector<int>>& arr2D, vector<int>& arr1D, int rows, int cols) {
6      int index = 0;
7      for (int j = 0; j < cols; ++j) {
8          for (int i = 0; i < rows; ++i) {
9              arr1D[index++] = arr2D[i][j];
10         }
11     }
12 }
13
14 int main() {
15     int rows, cols;
16     cout << "Enter the number of rows: ";
17     cin >> rows;
18     cout << "Enter the number of columns: ";
19     cin >> cols;
20
21     vector<vector<int>> arr2D(rows, vector<int>(cols));
22     cout << "Enter the elements of the 2D array (" << rows << "x" << cols << "):\n";
23     for (int i = 0; i < rows; ++i) {
24         for (int j = 0; j < cols; ++j) {
25             cin >> arr2D[i][j];
26         }
27     }
28
29     vector<int> arr1D(rows * cols);
30
31     // Call the function
32     convert2D(arr2D, arr1D, rows, cols);
33
34     // Display the resulting 1D array
35     cout << "1D Array in Column Major Order: ";
36     for (int i = 0; i < rows * cols; ++i) {
37         cout << arr1D[i] << " ";
38     }
39     cout << endl;
40
41     return 0;
}
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\Users\User\OneDrive\Desktop\DSA\" ; if ($?) { g++ ques1.cpp -o ques1 } ; if ($?) { .\ques1 }
Enter the number of rows: 3
Enter the number of columns: 2
Enter the elements of the 2D array (3x2):
1
45
2
4
77
6
1D Array in Column Major Order: 1 2 77 45 4 6
```

QUESTION 2:

SOURCE CODE:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void calculateAndDisplayGPAs(string students[], double grades[][5], int numStudents, int numSubjects, int creditHours) {
5      cout << fixed << setprecision(2);
6      cout << "--- Semester GPA for Students ---\n\n";
7
8      for (int i = 0; i < numStudents; i++) {
9          double totalWeightedPoints = 0.0;
10         int totalCredits = 0;
11
12         for (int j = 0; j < numSubjects; j++) {
13             // Check if the student took the course (-1.0 means course was skipped)
14             if (grades[i][j] != -1.0) {
15                 totalWeightedPoints += grades[i][j] * creditHours;
16                 totalCredits += creditHours;
17             }
18         }
19
20         // Calculate GPA: Sum of (Grade * Credits) / Total Credits
21         double gpa = (totalCredits > 0) ? (totalWeightedPoints / totalCredits) : 0.0;
22
23         cout << "Student: " << setw(8) << left << students[i] << " | GPA: " << gpa << endl;
24     }
25 }
26
27 int main() {
28     const int numStudents = 5;
29     const int numSubjects = 5;
30
31     string students[numStudents] = {"Ali", "Hiba", "Asma", "Zain", "Faisal"};
32
33     double grades[numStudents][numSubjects] = {
34         {3.66, 3.33, 4.00, 3.00, 2.66}, // Ali
35         {3.33, 3.00, 3.66, 3.00, -1.0}, // Hiba
36         {4.00, 3.66, 2.66, -1.0, -1.0}, // Asma
37         {2.66, 2.33, 4.00, -1.0, -1.0}, // Zain
38         {3.33, 3.66, 4.00, 3.00, 3.33} // Faisal
39     };
40
41     int creditHours = 3;
42
43     calculateAndDisplayGPAs(students, grades, numStudents, numSubjects, creditHours);
44
45     return 0;
46 }
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\Users\User\OneDrive\Desktop\DSA\" ; if ($?) { g++ ques2.cpp -o ques2 } ; if ($?) { .\ques2 }  
--- Semester GPA for Students ---  
  
Student: Ali      | GPA: 3.33  
Student: Hiba     | GPA: 3.25  
Student: Asma     | GPA: 3.44  
Student: Zain     | GPA: 3.00  
Student: Faisal   | GPA: 3.46
```

QUESTION NO 3:

SOURCE

CODE:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class MedianFinder {
5  private:
6      vector<int> nums;
7
8  public:
9      MedianFinder() {}
10
11     void addNum(int num) {
12         nums.push_back(num);
13         sort(nums.begin(), nums.end());
14     }
15
16     double findMedian() {
17         int n = nums.size();
18         if (n % 2 != 0) {
19             // If odd, return the middle element
20             return nums[n / 2];
21         } else {
22             // If even, return the average of the two middle elements
23             return (nums[(n / 2) - 1] + nums[n / 2]) / 2.0;
24         }
25     }
26 };
27
28 int main() {
29     MedianFinder medianFinder;
30
31     medianFinder.addNum(1);
32     medianFinder.addNum(2);
33     cout << "Median: " << medianFinder.findMedian() << endl;
34
35     medianFinder.addNum(3);
36     cout << "Median: " << medianFinder.findMedian() << endl;
37
38     return 0;
39 }
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\Users\User\OneDrive\Desktop\DSA\" ; if ($?) { g++ ques3.cpp -o ques3 } ;
Median: 1.5
Median: 2
PS C:\Users\User\OneDrive\Desktop\DSA> 
```

QUESTION NO 4:

SOURCE CODE:

```
binary_search.cpp > ...
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int search(const vector<int>& nums, int target) {
6      int left = 0;
7      int right = nums.size() - 1;
8
9      while (left <= right) {
10         int mid = left + (right - left) / 2;
11
12         if (nums[mid] == target) {
13             return mid;
14         } else if (nums[mid] < target) {
15             left = mid + 1;
16         } else {
17             right = mid - 1;
18         }
19     }
20     return -1;
21 }
22
23 int main() {
24     vector<int> nums = {-1, 0, 3, 5, 9, 12};
25     int target = 9;
26
27     int result = search(nums, target);
28
29     if (result == -1) {
30         cout << "Element is not present in array" << endl;
31     } else {
32         cout << "Element is present at index " << result << endl;
33     }
34
35     return 0;
36 }
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\User
Element is present at index 4
PS C:\Users\User\OneDrive\Desktop\DSA>
```

QUESTION NO 5:

SOURCE CODE:

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  bool searchMatrix(const vector<vector<int>>& matrix, int target) {
6      if (matrix.empty() || matrix[0].empty()) {
7          return false;
8      }
9
10     int m = matrix.size();
11     int n = matrix[0].size();
12
13     int left = 0;
14     int right = m * n - 1;
15
16     while (left <= right) {
17         int mid = left + (right - left) / 2;
18         int midValue = matrix[mid / n][mid % n];
19
20         if (midValue == target) {
21             return true;
22         } else if (midValue < target) {
23             left = mid + 1;
24         } else {
25             right = mid - 1;
26         }
27     }
28
29     return false;
30 }
31
32 int main() {
33     vector<vector<int>> matrix = {
34         {1, 3, 5, 7},
35         {10, 11, 16, 20},
36         {23, 30, 34, 60}
37     };
38     int target = 3;
39
40     bool result = searchMatrix(matrix, target);
41
42     cout << (result ? "true" : "false") << endl;
43
44     return 0;
45 }
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\
true
PS C:\Users\User\OneDrive\Desktop\DSA>
```